TP 6 : Application Android – Météo

Développement d'applications mobiles

1 Introduction

Dans ce TP, nous allons développer une application mobile Android permettant à l'utilisateur de saisir le nom d'une ville et d'afficher les prévisions météorologiques correspondantes en utilisant une **API REST**. Les données seront récupérées depuis le service **OpenWeather** et traitées sous forme de **JSON**.

Ce TP a pour objectif principal de vous initier à la communication entre une application mobile et un serveur distant via le protocole **HTTP**, ainsi qu'à la manipulation de données structurées.

2 Objectifs pédagogiques

À la fin de ce TP, vous serez capables de :

- Comprendre le principe d'une API REST
- Envoyer des requêtes HTTP depuis une application Android
- Traiter des réponses au format JSON
- Utiliser la bibliothèque Volley
- Créer un modèle de données
- Implémenter un Adapter personnalisé
- Afficher dynamiquement des données dans un ListView
- Structurer un projet Android professionnel
- Publier un projet sur GitHub

3 Prérequis

4 Liens utiles

- Documentation Volley : https://developer.android.com/training/volley
- API OpenWeather : https://openweathermap.org/api

5 Présentation de l'API REST

Une API REST (Representational State Transfer) est une interface qui permet à une application cliente (mobile, web, desktop) de communiquer avec un serveur distant afin d'échanger des données.

Dans notre cas :

— Le client : notre application Android
— Le serveur : OpenWeather
— Le protocole : HTTP
— Le format des données : JSON

Exemple de requête

`https://samples.openweathermap.org/data/2.5/forecast?q=casablanca&appid=b6907d289e10d71`

Partie 1 : Création du projet Android

1. Cliquer sur **New Project**
2. Choisir **Empty views Activity**
3. Nom du projet : `TP 6 - MeteoApp`
4. Langage : Java
5. Minimum SDK : API 21

Partie 2 : Configuration avec Kotlin DSL

Dans ce TP, nous utiliserons exclusivement le format **Kotlin DSL** pour la configuration Gradle.

Ajout de Volley

Dans le fichier `build.gradle.kts` (Module : app) :

```
dependencies {
implementation("com.android.volley:volley:1.1.0")
}
```

Cliquez ensuite sur **Sync Now**.

Partie 3 : Autorisation Internet

Dans Android, l'accès à certaines ressources sensibles (Internet, caméra, stockage, localisation, etc.) est protégé par un système de permissions. Une application ne peut pas utiliser ces ressources sans l'autorisation explicite de l'utilisateur.

Dans notre cas, l'application doit accéder à Internet afin de pouvoir communiquer avec l'API OpenWeather et récupérer les données météorologiques.

Pourquoi une permission est-elle nécessaire ?

Lorsque votre application envoie une requête HTTP vers un serveur distant, elle utilise la connexion réseau de l'appareil. Android considère cela comme une action sensible, car elle peut :

— Consommer des données mobiles
— Mettre en danger la sécurité de l'utilisateur

— Accéder à des services externes

C'est pour cette raison qu'Android impose de déclarer explicitement cette permission dans le fichier `AndroidManifest.xml`.

Modification du fichier AndroidManifest.xml

Ouvrez le fichier `AndroidManifest.xml` et ajoutez la ligne suivante :

```
<uses-permission android:name="android.permission.INTERNET" />
```

Votre fichier peut ressembler à ceci :

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
<uses-permission android:name="android.permission.INTERNET" />
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.MeteoApp">
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER"
                />
        </intent-filter>
    </activity>
</application>
```

Questions de réflexion

1. Que se passerait-il si cette permission n'était pas ajoutée ?
2. Pourquoi Android ne permet-il pas l'accès automatique à Internet ?

Partie 4 : Conception de l'interface

Dans cette partie, nous allons concevoir l'interface graphique de notre application météo. Cette interface doit être simple, claire et intuitive pour l'utilisateur.

Elle sera composée de :

— Un champ de saisie pour entrer le nom de la ville

— Un bouton pour lancer la recherche

— Une liste pour afficher les prévisions météo

Organisation générale

L'interface principale sera définie dans le fichier `activity_main.xml`. Ce fichier décrit l'agencement des composants graphiques.

Nous utiliserons un `LinearLayout` avec une orientation verticale pour organiser les éléments les uns sous les autres.

Fichier `activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
   android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity" android:padding="10dp">
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <EditText
        android:id="@+id/editTextVille"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="Entrez␣une␣ville"
        android:inputType="textPersonName" />
    <ImageButton
        android:id="@+id/buttonOK"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@android:drawable/ic_media_play"
        android:contentDescription="Bouton␣de␣recherche" />
</LinearLayout>
<ListView
    android:id="@+id/listViewMeteo"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
</LinearLayout>
```

Explication des composants

- **LinearLayout** : permet d'organiser les composants les uns sous les autres ou côte à côte.
- **EditText** : permet à l'utilisateur de saisir le nom de la ville.
- **ImageButton** : bouton illustré par une icône pour lancer la recherche.
- **ListView** : affiche une liste de résultats dynamiques.

— Les composants utilisant `app:srcCompat` proviennent de la bibliothèque **AndroidX AppCompat**. Cette bibliothèque fournit des composants prêts à l'emploi compatibles avec les anciennes et nouvelles versions d'Android, afin d'assurer la compatibilité et éviter les erreurs d'affichage.

Fichier `list_item_layout.xml`

Chaque ligne de la liste représentera une prévision météo. Nous allons donc créer un layout spécifique pour chaque élément.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_vertical"
        tools:srcCompat="@tools:sample/avatars" />
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="10dp">
    <TextView
            android:id="@+id/textViewDate"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.AppCompat.
               Medium"
            android:textStyle="bold" />
    <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">
            <TextView
                android:id="@+id/textView"
                android:layout_width="93dp"
                android:layout_height="wrap_content"
                android:text="Min:"
                android:textAlignment="textStart"
                android:textAppearance="@style/TextAppearance.
                   AppCompat.Medium"
                />
            <TextView
                android:id="@+id/textViewTempMin"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
```

```
            android:layout_weight="1"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.
               AppCompat.Medium" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:id="@+id/textView4"
            android:layout_width="93dp"
            android:layout_height="match_parent"
            android:text="MAX:"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.
               AppCompat.Medium" />
        <TextView
            android:id="@+id/textViewTempMAX"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.
               AppCompat.Medium" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:id="@+id/textView6"
            android:layout_width="93dp"
            android:layout_height="wrap_content"
            android:text="Pression:"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.
               AppCompat.Medium" />
        <TextView
            android:id="@+id/textViewPression"
            android:layout_width="166dp"
            android:layout_height="wrap_content"
            android:textAlignment="textStart"
            android:textAppearance="@style/TextAppearance.
               AppCompat.Medium" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:id="@+id/textView9"
            android:layout_width="93dp"
```

```
                android:layout_height="wrap_content"
                android:text="Humidité:"
                android:textAlignment="textStart"
                android:textAppearance="@style/TextAppearance.
                   AppCompat.Medium" />

            <TextView
                android:id="@+id/textViewHumidite"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textAlignment="textStart"
                android:textAppearance="@style/TextAppearance.
                   AppCompat.Medium" />
        </LinearLayout>
    </LinearLayout>
</LinearLayout>
```

Travail demandé

- Créer les deux fichiers XML : `activity_main.xml` et `list_item_layout.xml`
- Vérifier que l'interface s'affiche correctement dans le mode Design
- Faire des captures d'écran de l'interface
- Ajouter les captures dans le rapport

Questions

1. Quel est le rôle du fichier `activity_main.xml` ?
2. Pourquoi utilise-t-on un layout séparé pour les éléments de la liste ?
3. Quelle est la différence entre un TextView et un EditText ?

Partie 5 : Modèle de données

Dans cette partie, nous allons créer une classe modèle en Java. Cette classe permettra de représenter une prévision météo sous forme d'un objet manipulable dans notre application.

Un modèle de données est une structure qui permet de stocker, organiser et transporter les informations provenant d'une source externe (ici, l'API OpenWeather).

Pourquoi utiliser une classe modèle ?

Lorsque nous récupérons des données depuis une API REST, celles-ci arrivent sous forme de texte (JSON). Pour pouvoir les manipuler facilement dans notre application, il est préférable de les convertir en objets Java.

L'utilisation d'une classe modèle permet de :

- Structurer les données
- Faciliter la lecture du code
- Améliorer la maintenabilité
- Simplifier l'affichage dans l'interface

Création de la classe MeteoItem

Créez une nouvelle classe Java nommée `MeteoItem`.

```
public class MeteoItem {
public int tempMax;
public int tempMin;
public int pression;
public int humidite;
public String image;
public String date; }
```

- **tempMax** : température maximale
- **tempMin** : température minimale
- **pression** : pression atmosphérique
- **humidite** : taux d'humidité
- **image** : type de météo (Clear, Clouds, Rain, etc.)
- **date** : date et heure de la prévision

Chaque fois que nous recevrons une nouvelle prévision météo depuis l'API, nous créerons un nouvel objet `MeteoItem` et nous le stockerons dans une liste.

Cette liste sera ensuite transmise à l'Adapter pour être affichée dans le `ListView`.

Questions de réflexion

1. Pourquoi ne pas utiliser directement le JSON dans l'interface ?
2. Quels sont les avantages de la programmation orientée objet dans ce cas ?

Partie 6 : Adapter personnalisé

Dans Android, un **Adapter** est un composant essentiel qui permet de faire le lien entre une source de données (par exemple une liste d'objets) et une vue graphique comme un `ListView`.

Dans notre application météo, chaque élément contient plusieurs informations : date, température, pression, humidité, icône, etc. Il est donc nécessaire d'utiliser un **Adapter personnalisé**.

Pourquoi un Adapter personnalisé ?

Un `ArrayAdapter` simple ne permet d'afficher qu'une seule chaîne de caractères par ligne.

Dans notre cas, chaque ligne doit contenir plusieurs champs et une mise en forme spécifique.

Principe de fonctionnement

Le rôle de l'Adapter est de :

- Recevoir une liste d'objets `MeteoItem`
- Associer chaque objet à une vue XML
- Remplir les composants graphiques avec les données
- Retourner la vue finale à afficher

Création de la classe MeteoListModel

Créez une nouvelle classe Java nommée `MeteoListModel`.

```
public class MeteoListModel extends ArrayAdapter<MeteoItem> {
    private List<MeteoItem> listItems; private int resource;
    public static Map<String,Integer> images=new HashMap<>();
    static{
        images.put("Clear",R.drawable.clear);
        images.put("Clouds",R.drawable.clouds);
        images.put("Rain",R.drawable.rain);
        images.put("thunderstormspng",R.drawable.thunderstorm);
    }
```

Le mot-clé `static` permet d'initialiser la table d'images une seule fois lors du chargement de la classe. Ainsi, la correspondance entre le type de météo et l'image est partagée par toutes les instances de la classe.

Ajoutez le constructeur :

```
public MeteoListModel(Context context, int resource, List<MeteoItem>
   data) {
    super(context, resource, data);
    this.listItems = data;
    this.resource = resource;
}

}
```

Méthode getView()

La méthode `getView()` est appelée pour chaque élément de la liste. Elle permet de personnaliser l'affichage.

```
@NonNull
@Override
public View getView(int position, @Nullable View convertView,
   @NonNull ViewGroup
            parent) {
View listItem = convertView;
if (listItem == null) {
listItem = LayoutInflater.from(getContext()).inflate(resource, parent
   , false); }

ImageView imageView=listItem.findViewById(R.id.imageView);
TextView textViewTempMax=listItem.findViewById(R.id.textViewTempMAX);
TextView textViewTempMin=listItem.findViewById(R.id.textViewTempMin);
TextView textViewPression=listItem.findViewById(R.id.textViewPression
   );
TextView textViewHumidite=listItem.findViewById(R.id.textViewHumidite
   );
TextView textViewDate=listItem.findViewById(R.id.textViewDate);
String key=listItems.get(position).image;
if(key!=null) imageView.setImageResource(images.get(key));
```

```
textViewTempMax.setText(String.valueOf(listItems.get(position).
   tempMax)+"␣ C ");
textViewTempMin.setText(String.valueOf(listItems.get(position).
   tempMin)+"␣ C ");
textViewPression.setText(String.valueOf(listItems.get(position).
   pression)+"␣hPa");
textViewHumidite.setText(String.valueOf(listItems.get(position).
   humidite)+"␣%");
textViewDate.setText(String.valueOf(listItems.get(position).date));
return listItem;

}
```

L'annotation `@NonNull` indique qu'une variable ou une valeur de retour ne doit jamais être `null`. Elle n'est pas obligatoire, mais permet d'éviter les erreurs et d'améliorer la qualité du code. Android Studio l'utilise pour détecter d'éventuels problèmes à la compilation.

Questions de réflexion

1. Quel est le rôle d'un Adapter dans Android ?
2. À quoi sert la méthode `getView()` ?
3. Pourquoi utilise-t-on un layout personnalisé pour chaque élément ?

Partie 7 : Appel de l'API avec Volley

Dans cette partie, nous allons connecter notre application Android à un service distant : l'API OpenWeather.

Pour cela, nous utiliserons la bibliothèque **Volley**, qui permet d'effectuer des requêtes HTTP facilement et efficacement.

Rappel : qu'est-ce qu'une API ?

Une API (Application Programming Interface) permet à deux applications de communiquer entre elles.

Dans notre cas :

— Notre application = client

— OpenWeather = serveur

— Protocole = HTTP

— Format des données = JSON

Pourquoi utiliser Volley ?

Volley est une bibliothèque Android qui permet :

— D'envoyer des requêtes HTTP (GET, POST, etc.)

— De gérer les réponses

— De traiter les erreurs

— De travailler de manière asynchrone

Étapes principales

1. Créer une RequestQueue
2. Construire l'URL de l'API
3. Créer une requête
4. Envoyer la requête
5. Traiter la réponse

Exemple de code Java avec Volley

```
String url = "https://api.openweathermap.org/data/2.5/forecast?q=
   Casablanca&appid=b6907d289e10d714a6e88b30761fae22";

StringRequest request = new StringRequest(Request.Method.GET, url,
   new Response.Listener() {
    @Override
    public void onResponse(String response) {
    // Traitement de la réponse JSON
    }
}, new Response.ErrorListener() {
    @Override
    public void onErrorResponse(VolleyError error) {
    // Gestion des erreurs
    }
});

queue.add(request);
```

Travail demandé

— Ajouter le code Volley dans votre activité principale
— Tester l'appel à l'API avec une ville
— Vérifier que la réponse est bien reçue
— Faire une capture d'écran du code
— Ajouter la capture dans le rapport

Questions de réflexion

1. À quoi sert la classe `RequestQueue` ?
2. Pourquoi les requêtes sont-elles asynchrones ?
3. Que se passe-t-il en cas d'erreur réseau ?

Partie 8 : Traitement et Parsing JSON

Dans cette partie, nous allons traiter les données reçues depuis l'API OpenWeather. Ces données sont fournies au format **JSON**.

Notre objectif est de :

— Lire la réponse JSON
— Extraire les informations utiles
— Créer des objets `MeteoItem`
— Stocker ces objets dans une liste

Rappel : Qu'est-ce que le JSON ?

JSON (JavaScript Object Notation) est un format léger d'échange de données, très utilisé dans les applications web et mobiles.

Il est basé sur deux structures principales :

— Les objets (JSONObject)
— Les tableaux (JSONArray)

Structure simplifiée de la réponse OpenWeather

La réponse de l'API contient plusieurs niveaux d'imbrication.

Nous allons principalement exploiter :

— La liste des prévisions
— Les températures
— L'humidité
— La pression
— La date

Exemple de code de parsing en Java

```
for (int i = 0; i < list.length(); i++) {
    JSONObject item = list.getJSONObject(i);
    JSONObject main = item.getJSONObject("main");

    int tempMin = (int) main.getDouble("temp_min");
    int tempMax = (int) main.getDouble("temp_max");
    int pression = main.getInt("pressure");
    int humidite = main.getInt("humidity");
    String date = item.getString("dt_txt");

    MeteoItem meteo = new MeteoItem();
    meteo.tempMin = tempMin;
    meteo.tempMax = tempMax;
    meteo.pression = pression;
    meteo.humidite = humidite;
    meteo.date = date;

    listeMeteo.add(meteo);
}
} catch (JSONException e) { e.printStackTrace(); }
```

Travail demandé

— Implémenter le parsing JSON dans votre projet
— Vérifier que les données sont bien extraites
— Afficher les données dans la console (Logcat)
— Faire une capture d'écran du code
— Ajouter la capture dans le rapport

Questions de réflexion

1. Quelle est la différence entre JSONObject et JSONArray ?
2. Pourquoi faut-il utiliser un bloc try/catch ?
3. Que se passe-t-il si une clé JSON n'existe pas ?

Partie 9 : Affichage des données

Dans cette partie, nous allons afficher les données météo traitées dans l'interface graphique à l'aide d'un `ListView` et de notre `Adapter personnalisé`.

Rappel du fonctionnement

L'affichage des données suit les étapes suivantes :

1. Récupération des données via l'API
2. Parsing JSON
3. Création des objets `MeteoItem`
4. Ajout dans une liste
5. Transmission à l'Adapter
6. Affichage dans le ListView

Initialisation du ListView

Dans votre activité principale, vous devez initialiser le ListView et l'Adapter.

```
MeteoListModel adapter = new MeteoListModel(this, R.layout.
   list_item_layout, listeMeteo); listView.setAdapter(adapter);
```

Mise à jour dynamique

Lorsque de nouvelles données sont ajoutées à la liste, l'interface ne se met pas à jour automatiquement.

Il est donc nécessaire d'appeler la méthode suivante :

```
adapter.notifyDataSetChanged();
```

Pourquoi notifyDataSetChanged() ?

Cette méthode permet d'informer l'Adapter que les données ont changé.
Sans cet appel :

- Les nouvelles données ne s'affichent pas
- L'interface reste inchangée
- L'utilisateur ne voit pas les résultats

Exemple d'intégration après parsing

```
listeMeteo.add(meteo);
adapter.notifyDataSetChanged();
```

Création de la classe MainActivity

```
package com.example.nomdevotreapplication;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.ListView;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.StringRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONArray;
import org.json.JSONException; import org.json.JSONObject;
import java.text.SimpleDateFormat; import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class MainActivity extends AppCompatActivity {
    private EditText editTextVille; private ListView listViewMeteo;
    //private List<String> data=new ArrayList<>();
    List<MeteoItem> data=new ArrayList<>();
    //private ArrayAdapter<String> model;
    private MeteoListModel model; private ImageButton buttonOK;
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        editTextVille=findViewById(R.id.editTextVille);
```

```
        listViewMeteo=findViewById(R.id.listViewMeteo);
        buttonOK=findViewById(R.id.buttonOK);
        model = new MeteoListModel(MainActivity.this,
                R.layout.list_item_layout, data);

        listViewMeteo.setAdapter(model);
//model=new ArrayAdapter<>(this,android.R.layout.simple_list_item_1,
   data);
        buttonOK.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Log.i("MyLog",".....");
                data.clear(); model.notifyDataSetChanged();
                RequestQueue queue = Volley.newRequestQueue(
                   getApplicationContext());
                String ville = editTextVille.getText().toString().
                   trim();
                if (ville.isEmpty()) {
                    editTextVille.setError("Entrer␣une␣ville");
                    return;
                }

                Log.i("MyLog",ville);
                String url = "https://api.openweathermap.org/data
                   /2.5/forecast?q="
                        + ville + "&appid=
                           a4578e39643716894ec78b28a71c7110";
                StringRequest stringRequest = new StringRequest(
                   Request.Method.GET, url,
                        new Response.Listener<String>() {
                            @Override
                            public void onResponse(String response) {
                                try {
                                    Log.i("MyLog","
                                       ----------------------------")
                                       ;
                                    Log.i("MyLog",response);
                                    List<MeteoItem> meteoItems=new
                                       ArrayList<>();
                                    JSONObject jsonObject=new
                                       JSONObject(response);
                                    JSONArray jsonArray=jsonObject.
                                       getJSONArray("list");
                                    for (int i=0;i<jsonArray.length()
                                       ;i++){
                                     MeteoItem meteoItem=new
                                        MeteoItem();
                                     JSONObject d=jsonArray.
                                        getJSONObject(i);
                                     Date date=new Date(d.getLong(
                                        "dt")*1000);
                                     SimpleDateFormat sdf = new
                                        SimpleDateFormat("dd-MMM-
```

```
                                                yyyy'T'HH:mm");
                                            String dateString = sdf.
                                                format(date);
                                            JSONObject main=d.
                                                getJSONObject("main");
                                            JSONArray weather=d.
                                                getJSONArray("weather");
                                            int tempMin=(int)(main.
                                                getDouble("temp_min")
                                                -273.15);
                                            int tempMax=(int)(main.
                                                getDouble("temp_max")
                                                -273.15);
                                            int pression=main.getInt("
                                                pressure");
                                            int humidity=main.getInt("
                                                humidity");
//data.add(String.format(dateString+"\n"+"Min=%d  c , Max=%d  C ",
    tempMin,tempMax));
                                            meteoItem.tempMax=tempMax;
                                                meteoItem.tempMin=tempMin;
                                            meteoItem.pression=pression;
                                                meteoItem.humidite=
                                                humidity;
                                            meteoItem.date=dateString;
                                            meteoItem.image=weather.
                                                getJSONObject(0).getString
                                                ("main");
                                            meteoItems.add(meteoItem);
                                            data.add(meteoItem);
                                        }
                                        model.notifyDataSetChanged();
                                    } catch (JSONException e) {
                                        e.printStackTrace();
                                    }
                                }
                            },
                            new Response.ErrorListener() {
                                @Override
                                public void onErrorResponse(VolleyError
                                    error) {
                                    Log.i("MyLog","Connection␣problem!");
                                }
                            });
// Add the request to the RequestQueue.
                queue.add(stringRequest);
            }
        });

    }
}
```

Travail demandé

- Initialiser correctement le ListView
- Associer l'Adapter personnalisé
- Afficher les données météo
- Tester avec plusieurs villes
- Faire des captures d'écran
- Ajouter les captures dans le rapport

Questions de réflexion

1. Quel est le rôle du ListView ?
2. À quoi sert notifyDataSetChanged() ?
3. Que se passe-t-il si on oublie cette méthode ?

Partie 10 : Cycle de vie de l'Activity

Dans cette partie, vous devez associer une **action précise** à chaque méthode du cycle de vie de l'Activity. Chaque action doit être choisie de manière cohérente avec l'état de l'application et avec les notions déjà étudiées (Toast, AlertDialog, Logcat, sauvegarde d'état).

Travail demandé

Dans l'activité principale, implémentez les méthodes du cycle de vie suivantes et réalisez l'action demandée pour chacune d'elles.

- `onStart()` Afficher un `Toast` indiquant que l'application devient visible à l'écran.
- `onResume()` Autoriser l'interaction utilisateur et afficher un `Toast` indiquant que l'application est prête à être utilisée.
- `onPause()` Afficher un `Toast` indiquant que l'application perd le focus et écrire un message dans le `Logcat` pour signaler la mise en pause.
- `onStop()` Afficher une `AlertDialog` informant l'utilisateur que l'application est désormais en arrière-plan.
- `onRestart()` Afficher une `AlertDialog` demandant à l'utilisateur s'il souhaite continuer l'utilisation de l'application ou annuler.
- `onDestroy()` Afficher un dernier message dans le `Logcat` indiquant la fermeture définitive de l'application et libérer les ressources utilisées si nécessaire.

Consigne importante

Chaque action doit être exécutée dans la méthode du cycle de vie correspondante, et doit être capturé pour la joindre dans le rapport. Toute action mal placée ou incohérente avec l'état de l'application sera pénalisée.

Consignes pour le rapport

À la fin de la séance, vous devez rendre un rapport structuré comme suit :

1. Page de garde
2. Introduction
3. Réponses aux questions demandées
4. Captures d'écran commentées
5. Conclusion

Lien vers le rapport à modifier : https://1drv.ms/w/c/bd61a6d4df3cf63d/IQCwmJcjJf3aRZLVOzv8V
e=tOPVOP

6 Dépôt GitHub

Votre projet doit être déposé sur GitHub avec la structure suivante :

— `/app`

— `/captures`

— `README.md`

— `Rapport.pdf` **Nom du Rapport :** `TP6_CycleDeVie_NomPrenom.pdf`

7 Soumission

Soumettez le lien GitHub via le formulaire suivant :
[https://forms.gle/N11t1gqa9mCgR7GM6]